Mediator Design Pattern (Behavioral)

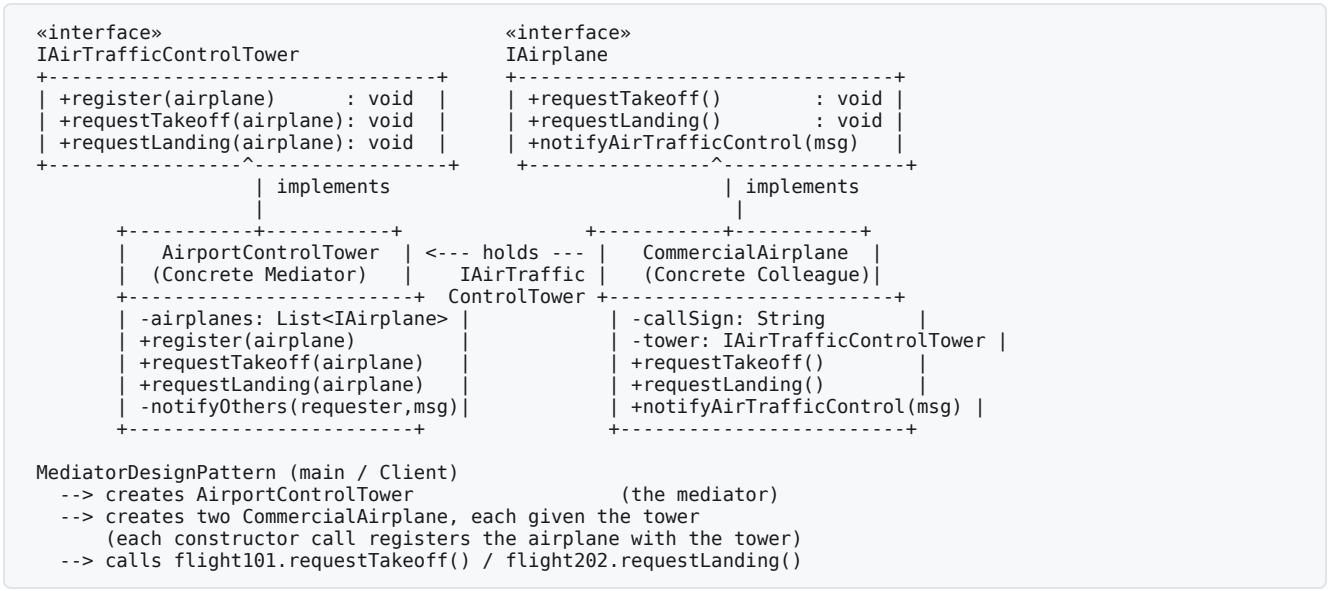
An air-traffic control tower coordinates every takeoff and landing so that individual aircraft never have to talk to one another directly.

Intent

The Mediator pattern defines an object that encapsulates *how* a set of other objects interact. Instead of each object holding references to every other object it needs to collaborate with (an N×M web of direct couplings), every object holds a single reference to the mediator, and the mediator decides how to route each interaction. This turns a tangled many-to-many relationship into a simple hub-and-spoke one, so individual collaborators can be developed, tested, and replaced without knowing about each other's existence.

This module is the **canonical GoF Mediator**: a `Mediator` interface (`IAirTrafficControlTower`) sits in front of the concrete mediator (`AirportControlTower`), and a `Colleague` interface (`IAirplane`) sits in front of the concrete colleagues (`CommercialAirplane`). Both sides of the collaboration are abstracted, which is what the full pattern — as opposed to a bare dispatcher — requires.

UML class diagram (ASCII)



Note the direction of the "holds" arrow: `CommercialAirplane` holds a reference to `IAirTrafficControlTower`. There is **no** arrow between the two `CommercialAirplane` instances — that absence is the whole point of the pattern.

The players

- **`IAirTrafficControlTower`** — the *Mediator interface*. Declares the contract every concrete mediator must expose: register a colleague, and route a takeoff/landing request from one.
- **`AirportControlTower`** — the *concrete Mediator*. Keeps the list of registered airplanes and contains all the coordination logic: granting the requester's clearance and notifying every other registered airplane.
- **`IAirplane`** — the *Colleague interface*. Declares what a colleague can be asked to do (`requestTakeoff`, `requestLanding`) and what the mediator can tell it (`notifyAirTrafficControl`).
- **`CommercialAirplane`** — the *concrete Colleague*. Holds its own `callSign` and a reference to the mediator (`IAirTrafficControlTower`) — **and nothing else**. It never references another `CommercialAirplane`.
- **`MediatorDesignPattern`** — the *client* / entry point. Wires one mediator and two colleagues together, then drives the demo through the colleague interface only.

Code walkthrough — every line explained

`IAirplane.java`

```
package com.design.patterns.mediator.colleague;

public interface IAirplane {

    void requestTakeoff();

    void requestLanding();

    void notifyAirTrafficControl(String msg);

}
```

- `package com.design.patterns.mediator.colleague;` — places the colleague contract in its own `colleague` package, separate from the `mediator` package, so the two sides of the pattern are physically as well as logically distinct.
- `public interface IAirplane {` — declares the Colleague interface as `public` so both the mediator package and the client package can depend on it. An interface (rather than an abstract class) is used because a colleague's only obligation is to honor this contract; it imposes no implementation inheritance.
- `void requestTakeoff();` — declares the operation a colleague exposes to trigger a takeoff request. It returns nothing because the real work (granting clearance, notifying peers) happens on the mediator side, not synchronously as a return value.
- `void requestLanding();` — mirrors `requestTakeoff()` for the landing scenario. Keeping the two symmetric keeps the contract easy to extend (e.g., a future `requestTaxi()`).
- `void notifyAirTrafficControl(String msg);` — declares the callback the mediator uses to push a message *into* the colleague. This is the method that makes the collaboration two-way: colleagues both request things of the mediator and receive notifications from it.
- `}` — closes the interface body.

`CommercialAirplane.java`

```
package com.design.patterns.mediator.colleague.concrete;

import com.design.patterns.mediator.colleague.IAirplane;
import com.design.patterns.mediator.mediator.IAirTrafficControlTower;

public class CommercialAirplane implements IAirplane {

    private final String callSign;
    private final IAirTrafficControlTower tower;

    public CommercialAirplane(String callSign, IAirTrafficControlTower tower) {
        this.callSign = callSign;
        this.tower = tower;
        tower.register(this);
    }

    @Override
    public void requestTakeoff() {
        System.out.println(callSign + " -> Tower: requesting takeoff.");
        tower.requestTakeoff(this);
    }

    @Override
    public void requestLanding() {
        System.out.println(callSign + " -> Tower: requesting landing.");
        tower.requestLanding(this);
    }

    @Override
    public void notifyAirTrafficControl(String msg) {
        System.out.println("Tower -> " + callSign + ": " + msg);
    }
}
```

- `package com.design.patterns.mediator.colleague.concrete;` — places this class in the `concrete` sub-package of `colleague`, separating the one concrete colleague implementation from the interface it implements. New colleague types (e.g., `CargoAirplane`) would live alongside it here without touching the interface.
- `import com.design.patterns.mediator.colleague.IAirplane;` — brings the Colleague interface into scope so this class can declare `implements IAirplane`.
- `import com.design.patterns.mediator.mediator.IAirTrafficControlTower;` — brings the *Mediator interface* (not the concrete `AirportControlTower`) into scope. This is the single most important import in the file: it means `CommercialAirplane` is coupled to the mediator's abstraction, never to a specific mediator implementation.
- `public class CommercialAirplane implements IAirplane {` — declares the concrete colleague. `implements IAirplane` is the compile-time guarantee that all three contract methods are provided. The opening brace begins the class body.

- `private final String callSign;` — the airplane's identity, used only for readable console output. `private` encapsulates it; `final` means an airplane's call sign never changes after construction.
- `private final IAirTrafficControlTower tower;` — the **only** collaborator reference this class holds. It is typed to the interface, not `AirportControlTower`, so this colleague would work unchanged against any mediator implementation. Critically, there is no field here referencing another `IAirplane` — colleagues in this pattern never know about each other.
- `public CommercialAirplane(String callSign, IAirTrafficControlTower tower) {` — the constructor takes the airplane's identity and the mediator it will collaborate through. Both are supplied by the client at construction time (constructor injection), keeping the class itself free of any lookup/registry logic.
- `this.callSign = callSign;` — stores the call sign field.
- `this.tower = tower;` — stores the mediator reference field.
- `tower.register(this);` — the constructor immediately registers itself with the mediator, passing `this` (the newly-constructed colleague) so the mediator can address it later. Doing this in the constructor guarantees a `CommercialAirplane` can never exist without also being known to the tower — there is no window where an unregistered airplane could request anything.
- `}` — closes the constructor body.
- `@Override` — verifies at compile time that `requestTakeoff` genuinely implements `IAirplane.requestTakeoff()`.
- `public void requestTakeoff() {` — opens the takeoff-request method.
- `System.out.println(callSign + " -> Tower: requesting takeoff.");` — logs the outbound request so the console trace makes the interaction visible.
- `tower.requestTakeoff(this);` — delegates entirely to the mediator, passing `this` so the mediator knows *which* airplane is requesting. This is the crux of the pattern: `CommercialAirplane` does not decide what happens next (whether other airplanes should be told, what clearance text to print) — it hands that decision to `AirportControlTower`.
- `}` — closes `requestTakeoff`.
- `@Override` — verifies `requestLanding` implements the interface method.
- `public void requestLanding() {` — opens the landing-request method, structurally identical to takeoff.
- `System.out.println(callSign + " -> Tower: requesting landing.");` — logs the outbound request.
- `tower.requestLanding(this);` — delegates to the mediator.
- `}` — closes `requestLanding`.
- `@Override` — verifies `notifyAirTrafficControl` implements the interface method.
- `public void notifyAirTrafficControl(String msg) {` — opens the inbound callback the mediator uses to push information to this colleague.
- `System.out.println("Tower -> " + callSign + ": " + msg);` — prints the message it was handed. This is the *only* place `CommercialAirplane` reacts to something happening elsewhere in the system, and it always arrives via the mediator, never directly from another airplane.
- `}` — closes `notifyAirTrafficControl`.
- `}` — closes the class body.

`IAirTrafficControlTower.java`

```
package com.design.patterns.mediator.mediator;

import com.design.patterns.mediator.colleague.IAirplane;

public interface IAirTrafficControlTower {

    void register(IAirplane airplane);

    void requestTakeoff(IAirplane airplane);

    void requestLanding(IAirplane airplane);

}
```

- `package com.design.patterns.mediator.mediator;` — the mediator's own package, mirroring the `colleague` package on the other side of the collaboration.
- `import com.design.patterns.mediator.colleague.IAirplane;` — brings the Colleague interface into scope, because every method on this interface takes an `IAirplane` parameter. The mediator depends on the colleague *abstraction*, never on `CommercialAirplane` directly.
- `public interface IAirTrafficControlTower {` — declares the Mediator interface. This is what makes the pattern the full GoF Mediator rather than a bare dispatcher class: the client and the colleagues can both be written against this abstraction, and a different concrete tower (e.g., a `MilitaryControlTower` with different rules) could be substituted with zero changes elsewhere.
- `void register(IAirplane airplane);` — declares how a colleague joins the collaboration. Every concrete mediator must provide a way for colleagues to make themselves known.
- `void requestTakeoff(IAirplane airplane);` — declares the takeoff-coordination entry point. The `IAirplane` parameter tells the mediator *who* is asking, without the mediator needing any other identifying mechanism.
- `void requestLanding(IAirplane airplane);` — declares the landing-coordination entry point, symmetric to takeoff.
- `}` — closes the interface body.

`AirportControlTower.java`

```
package com.design.patterns.mediator.mediator.concrete;

import java.util.ArrayList;
import java.util.List;

import com.design.patterns.mediator.colleague.IAirplane;
import com.design.patterns.mediator.mediator.IAirTrafficControlTower;

public class AirportControlTower implements IAirTrafficControlTower {

    private final List<IAirplane> airplanes = new ArrayList<>();

    @Override
    public void register(IAirplane airplane) {
        airplanes.add(airplane);
    }

    @Override
    public void requestTakeoff(IAirplane airplane) {
        airplane.notifyAirTrafficControl("Takeoff clearance granted.");
        notifyOthers(airplane, "Hold position: another aircraft is taking off.");
    }

    @Override
    public void requestLanding(IAirplane airplane) {
        airplane.notifyAirTrafficControl("Landing clearance granted.");
        notifyOthers(airplane, "Stay clear of the runway: another aircraft is landing.");
    }

    private void notifyOthers(IAirplane requester, String msg) {
        airplanes.stream()
                .filter(other -> other != requester)
                .forEach(other -> other.notifyAirTrafficControl(msg));
    }

}
```

- `package com.design.patterns.mediator.mediator.concrete;` — places the one concrete mediator implementation in a `concrete` sub-package of `mediator`, mirroring the `colleague.concrete` layout on the other side. This keeps "the contract" and "the one implementation of the contract" visually and physically separated.
- `import java.util.ArrayList;` / `import java.util.List;` — bring in the collection types needed to hold the registered colleagues.
- `import com.design.patterns.mediator.colleague.IAirplane;` — brings the Colleague interface into scope; the tower stores and addresses colleagues only through this abstraction, never through `CommercialAirplane` directly.
- `import com.design.patterns.mediator.mediator.IAirTrafficControlTower;` — brings the Mediator interface into scope so this class can declare `implements IAirTrafficControlTower`.
- `public class AirportControlTower implements IAirTrafficControlTower {` — declares the concrete mediator. `implements IAirTrafficControlTower` forces all three contract methods to be provided.
- `private final List<IAirplane> airplanes = new ArrayList<>();` — the mediator's central state: every registered colleague, in registration order. This is the *only* place in the whole module that holds references to more than one colleague at a time — which is exactly why colleagues don't need to hold each other.
- `@Override public void register(IAirplane airplane) {` — implements registration.
- `airplanes.add(airplane);` — appends the newly-registered colleague to the list. After this call the tower can address this airplane whenever coordination requires it.
- `}` — closes `register`.
- `@Override public void requestTakeoff(IAirplane airplane) {` — implements the takeoff-coordination entry point.
- `airplane.notifyAirTrafficControl("Takeoff clearance granted.");` — the tower calls back into the requester first, granting clearance. This is a direct method call on the interface the tower already holds a reference to (because the requester passed `this` when it called `requestTakeoff`).
- `notifyOthers(airplane, "Hold position: another aircraft is taking off.");` — delegates to the private helper to inform every *other* registered colleague. The requesting airplane itself is excluded.
- `}` — closes `requestTakeoff`.
- `@Override public void requestLanding(IAirplane airplane) {` — implements the landing-coordination entry point, structurally identical to takeoff but with different messages.
- `airplane.notifyAirTrafficControl("Landing clearance granted.");` — grants landing clearance to the requester.
- `notifyOthers(airplane, "Stay clear of the runway: another aircraft is landing.");` — warns every other registered colleague.
- `}` — closes `requestLanding`.
- `private void notifyOthers(IAirplane requester, String msg) {` — a private helper shared by both `requestTakeoff` and `requestLanding`, avoiding duplicated iteration logic. `private` because this is an internal coordination detail, not part of the public mediator contract.
- `airplanes.stream()` — begins a stream over every registered colleague.
- `.filter(other -> other != requester)` — excludes the requester itself (identity comparison, not `equals`, since two colleagues are only "the same" if they are literally the same object) so it does not receive its own hold/clear-runway warning.
- `.forEach(other -> other.notifyAirTrafficControl(msg));` — pushes the message to every remaining colleague through the `IAirplane` interface. This is the one place where the tower "broadcasts" — and it is only possible because the tower, not any individual colleague, holds the full list.

- `}` — closes `notifyOthers`.
- `}` — closes the class body.

`MediatorDesignPattern.java`

```
package com.design.patterns.mediator;

import com.design.patterns.mediator.colleague.IAirplane;
import com.design.patterns.mediator.colleague.concrete.CommercialAirplane;
import com.design.patterns.mediator.mediator.IAirTrafficControlTower;
import com.design.patterns.mediator.mediator.concrete.AirportControlTower;

public class MediatorDesignPattern {

    public static void main(String[] args) {
        System.out.println("Mediator Design Pattern");

        IAirTrafficControlTower controlTower = new AirportControlTower();

        IAirplane flight101 = new CommercialAirplane("Flight-101", controlTower);
        IAirplane flight202 = new CommercialAirplane("Flight-202", controlTower);

        flight101.requestTakeoff();
        flight202.requestLanding();
    }
}
```

- `package com.design.patterns.mediator;` — the root package for this module, one level above both `colleague` and `mediator`. The client sits above both sides of the pattern, which is appropriate since it is the only piece of code allowed to know about *both* interfaces and both concrete classes.
- `import com.design.patterns.mediator.colleague.IAirplane;` — brings the Colleague interface into scope so local variables can be declared against the abstraction rather than `CommercialAirplane`.
- `import com.design.patterns.mediator.colleague.concrete.CommercialAirplane;` — brings the one concrete colleague class into scope. This import is required because the client is the only place a concrete colleague is ever `new`'d.
- `import com.design.patterns.mediator.mediator.IAirTrafficControlTower;` — brings the Mediator interface into scope for the same reason — declaring the local variable against the abstraction.
- `import com.design.patterns.mediator.mediator.concrete.AirportControlTower;` — brings the one concrete mediator class into scope; again, the client is the only place it is constructed.
- `public class MediatorDesignPattern {` — declares the demo/driver class. `public` is required so the JVM launcher can find `main` by class name.
- `public static void main(String[] args) {` — the JVM entry point. `public` so the JVM can invoke it without an instance; `static` because no `MediatorDesignPattern` object needs to exist first; `String[] args` is unused here.
- `System.out.println("Mediator Design Pattern");` — prints a header line identifying which pattern's demo is running, useful when several pattern demos are chained in a build pipeline.
- `IAirTrafficControlTower controlTower = new AirportControlTower();` — constructs the one concrete mediator, but stores the reference in a variable typed to the **interface**. From this line on, nothing else in the file (or in `CommercialAirplane`) ever needs to know the concrete class `AirportControlTower` exists.
- `IAirplane flight101 = new CommercialAirplane("Flight-101", controlTower);` — constructs the first colleague, injecting the mediator. Inside the constructor, `tower.register(this)` runs immediately, so by the time this line finishes, the tower already knows about `flight101`. The variable is typed to `IAirplane`, not `CommercialAirplane`.
- `IAirplane flight202 = new CommercialAirplane("Flight-202", controlTower);` — constructs the second colleague the same way, registering it with the same tower instance. Note that `flight101` and `flight202` never receive a reference to each other — the only shared object is `controlTower`.
- `flight101.requestTakeoff();` — invokes the request purely through the `IAirplane` interface. Internally this triggers `AirportControlTower.requestTakeoff`, which grants `flight101` clearance and then warns `flight202` (the only *other* registered airplane) to hold position.
- `flight202.requestLanding();` — invokes the landing request through the `IAirplane` interface. Internally this grants `flight202` clearance and warns `flight101` to stay clear of the runway.
- `}` — closes `main`.
- `}` — closes the class body.

Why these design decisions

Why two interfaces (`IAirTrafficControlTower` and `IAirplane`) instead of one?

The Mediator pattern has two distinct roles — the coordinator and the participants — and GoF models both as separate abstractions. Splitting them lets either side vary independently: a `CargoAirplane` colleague could be added without touching the tower, and a `MilitaryControlTower` mediator with stricter rules could be substituted without touching any airplane class. A single shared interface would blur these two responsibilities together.

Why does `CommercialAirplane` register itself in its constructor rather than the client registering it?

Doing registration inside the constructor makes "constructed" and "known to the mediator" the same moment — there is no way to end up with an airplane object that exists but that the tower doesn't know about. This removes a whole class of bugs where a caller forgets a separate `tower.register(...)` step.

Why does `notifyOthers` live on the mediator, not on each colleague?

Broadcasting to "every other" colleague requires knowing the full set of colleagues. Only the mediator holds that full list (`airplanes`); giving each colleague its own copy of that list would recreate the N×M coupling the pattern exists to eliminate, and would require every colleague to be updated whenever the roster changes.

Why type every reference to the interface, never the concrete class?

`controlTower` , `flight101` , and `flight202` in `MediatorDesignPattern` are all declared with interface types, and `CommercialAirplane` 's `tower` field is typed `IAirTrafficControlTower` . This is what keeps the pattern genuinely substitutable: any code written against `IAirplane` / `IAirTrafficControlTower` continues to work if `AirportControlTower` or `CommercialAirplane` are swapped for different implementations, as long as the new classes honor the same contracts.

Trade-offs

Aspect	Benefit	Cost
Registration in the constructor	Impossible to have an unregistered colleague	Colleague objects cannot be constructed "detached" for later wiring/testing
Mediator owns the full colleague list	Single source of truth for broadcast, colleagues stay lightweight	Mediator becomes the one class that must scale if the colleague count grows large
Two separate interfaces	Either side can vary independently	Slightly more files/indirection than a single concrete pair
`List<IAirplane>` (not a `Set`)	Preserves registration order for deterministic notification order	No automatic de-duplication if the same colleague is (mis)registered twice

---

Execution flow (step-by-step trace of what happens when `main()` runs)

1. `main` begins; `"Mediator Design Pattern"` is printed.
2. `new AirportControlTower()` constructs the mediator with an empty `airplanes` list. The reference is stored in `controlTower` (typed `IAirTrafficControlTower`).
3. `new CommercialAirplane("Flight-101", controlTower)` :
4. `callSign = "Flight-101"` , `tower = controlTower` are stored.
5. `tower.register(this)` runs → `AirportControlTower.airplanes` becomes `[Flight-101]` .
6. The reference is stored in `flight101` (typed `IAirplane`).
7. `new CommercialAirplane("Flight-202", controlTower)` :
8. `callSign = "Flight-202"` , `tower = controlTower` are stored.
9. `tower.register(this)` runs → `AirportControlTower.airplanes` becomes `[Flight-101, Flight-202]` .
10. The reference is stored in `flight202` .
11. `flight101.requestTakeoff()` :
12. Prints `Flight-101 -> Tower: requesting takeoff.`
13. Calls `tower.requestTakeoff(flight101)` .
14. Inside `AirportControlTower.requestTakeoff` : `flight101.notifyAirTrafficControl("Takeoff clearance granted.")` → prints `Tower -> Flight-101: Takeoff clearance granted.`
15. `notifyOthers(flight101, "Hold position: another aircraft is taking off.")` filters out `flight101` , leaving `[Flight-202]` , and calls `notifyAirTrafficControl` on it → prints `Tower -> Flight-202: Hold position: another aircraft is taking off.`
16. `flight202.requestLanding()` :
17. Prints `Flight-202 -> Tower: requesting landing.`
18. Calls `tower.requestLanding(flight202)` .
19. Inside `AirportControlTower.requestLanding` : `flight202.notifyAirTrafficControl("Landing clearance granted.")` → prints `Tower -> Flight-202: Landing clearance granted.`
20. `notifyOthers(flight202, "Stay clear of the runway: another aircraft is landing.")` filters out `flight202` , leaving `[Flight-101]` , and calls `notifyAirTrafficControl` on it → prints `Tower -> Flight-101: Stay clear of the runway: another aircraft is landing.`
21. `main` returns; the JVM exits with code 0.

At no point does `flight101` or `flight202` call a method on each other — every interaction is mediated by `controlTower` .

Expected output

```
Mediator Design Pattern
Flight-101 -> Tower: requesting takeoff.
Tower -> Flight-101: Takeoff clearance granted.
Tower -> Flight-202: Hold position: another aircraft is taking off.
Flight-202 -> Tower: requesting landing.
Tower -> Flight-202: Landing clearance granted.
Tower -> Flight-101: Stay clear of the runway: another aircraft is landing.
```

How to run

```
# From the module root: Mediator_Design_Pattern/
# The environment's default JDK breaks Lombok elsewhere in this reactor,
# so build with Java 11 explicitly:

JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64 mvn -o clean compile
java -cp target/classes com.design.patterns.mediator.MediatorDesignPattern

# Without the offline flag (downloads dependencies if needed):
JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64 mvn clean compile
java -cp target/classes com.design.patterns.mediator.MediatorDesignPattern
```

Other real-world problems this pattern can solve

1. **Air traffic control / logistics coordination** (this example) — a central coordinator sequences requests from many independent actors that must never negotiate directly with each other.
2. **Chat room server** — a `ChatRoom` mediator relays messages between `User` colleagues; no `User` object ever holds a reference to another `User`.
3. **GUI dialog coordination** — a dialog-box mediator enables/disables buttons and fields in response to other widgets changing state, so widgets don't wire directly to one another.
4. **Air-traffic-style resource locking** — any system where many workers must request exclusive access to a shared resource (a runway, a database row, a robotic arm) through one arbiter that also has to warn everyone else.
5. **Workflow orchestration** — a saga/orchestrator coordinates multiple services' steps and compensations without any two services calling each other directly.